Forthright: 3rd Person Shooter

Χρήστος Ρίγγας

Τεχνών Ήχου και Εικόνας, Ιόνιο Πανεπιστήμιο, Κέρκυρα Ελλάδα

Λέξεις Κλειδιά
Ανάπτυξη παιχνιδιού, παιχνίδια σκοποβολής, παιχνίδι τρίτου προσώπου, ανταγωνιστικό παιχνίδι, συλλογή πόντων, τρισδιάστατα περιβάλλοντα, διαχωρισμένη οθόνη, Input System, Unity Game Engine

1. Εισαγωγή

Η παρούσα εργασία πραγματεύεται την ανάπτυξη ενός τρισδιάστατου παιχνιδιού σκοποβολής τρίτου προσώπου χρησιμοποιώντας την μηχανή παιχνιδιών Unity Game Engine. Η Εφαρμογή/ παιχνίδι παίζεται αποκλειστικά με δύο παίκτες με τον κάθε παίκτη να έχει την δική του οπτική, εφόσον το παιχνίδι κατατάσσεται στην κατηγορία των παιχνιδιών διαχωρισμένης οθόνης. Οι δύο παίκτες ανταγωνίζονται μεταξύ τους με σκοπό να συλλέξουν τους περισσότερους πόντους. Οι πόντοι άλλοτε συλλέγονται με την εξόντωση του άλλου παίκτη και άλλοτε με την εύστοχη βολή των στόχων σύμφωνα με την επιλεγμένη λειτουργία στην οποία βρίσκονται. Δηλαδή: Α) η λειτουργεία “Practice” στην οποία σκοπός είναι οι παίκτες να καταφέρουν να πέτυχουν έναν στόχο, ο οποίος αφού χτυπηθεί αλλάζει τοποθεσία με τυχαίο τρόπο. Στην λειτουργία αυτή παρόλα αυτά, μπορούν να κάνουν εξάσκηση πετυχαίνοντας και άλλα αντικείμενα στον χώρο αγνοώντας το ανταγωνιστικό στοιχείο της πίστας. Β) η λειτουργεία “Versus” στην οποία οι παίκτες ανταγωνίζονται προσπαθώντας να εξοντώσουν ο ένας τον άλλον μαζεύοντας πόντους. Μεταξύ άλλον, στην σχεδίαση του παιχνιδιού δόθηκε ιδιαίτερη έμφαση στην λειτουργικότητα του με διαφορετικές συσκευές, κάνοντας χρήση της βιβλιοθήκης Input System [6] της μηχανής παιχνιδιών Unity. Οπότε, ο ένας παίκτης μπορεί να ελέγχει τον χαρακτήρα του με πληκτρολόγιο και ποντίκι, ενώ ο άλλος να κάνει χρήση ενός gamepad. Ενώ είναι ελεύθεροι να επιλέξουν, συνιστάται η χρήση gamepads και από τους δύο, ώστε να είναι ισορροπημένη η δυσκολία του παιχνιδιού και για του δύο παίκτες.

Σύνδεσμος Αρχείων Εργασίας:

2. Σχεδίαση Παιχνιδιού

Στόχος

Η σχεδίαση του παιχνιδιού αυτού έγινε με σκοπό την μελέτη του Input System της μηχανής Unity και την υλοποίηση ενός 3rd person shooter με βάση αυτό το σύστημα. Κύριος στόχος είναι η ανάπτυξη ενός απλουστευμένου ανταγωνιστικού παιχνιδιού μεταξύ δύο παικτών, όπου ο πιο εύστοχος νικάει. Επιπρόσθετα, η λειτουργία διαχωρισμού οθόνης ήταν στους βασικούς στόχους, ώστε να μπορούν οι παίκτες να ελέγχουν τους χαρακτήρες τους από διαφορετικές οπτικές/ κάμερες ανάλογα με την τοποθεσία τους στον χώρο. Αυτό γιατί δίνεται η δυνατότητα στους χαρακτήρες να βρίσκονται σε μεγάλες αποστάσεις μεταξύ τους χωρίς να χρειάζεται να περιοριστούν στην ίδια οπτική που προσφέρει η μία κάμερα. Τέλος, ο χειρισμός των διαφόρων μενού εντός του παιχνιδιού από όλες τις συσκευές αποτέλεσε σχεδιαστικός στόχος.

Gameplay

Ως ένα 3rd Person Shooter διαθέτει μερικές από τις βασικές λειτουργείες των παιχνιδιών της κατηγορίας. Δηλαδή, συνδυαστικά με animations πάνω στον χαρακτήρα,

ο παίκτης είναι σε θέση να κινηθεί στον τρισδιάστατο χώρο, να κοιτάξει γύρο του, να αναπηδήσει πάνω σε αντικείμενα, να πυροβολήσει αντικείμενα και χαρακτήρες με ένα προκαθορισμένο όπλο που του δίνεται, επίσης μπορεί να οπλίσει το όπλο αυτό σε περίπτωση που του τελειώσουν οι σφαίρες και τέλος έχει την δυνατότητα να συλλέγει αντικείμενα τα οποία είτε τον θεραπεύουν γεμίζοντας τους πόντους ζωής του, είτε του αναπληρώνουν τους γεμιστήρες του όπλου δίνοντάς του την δυνατότητα να το ξανά οπλίσει.

Κατά την διάρκεια της μάχης δεν υπάρχει κάποιος χρονικός περιορισμός ή ένα προκαθορισμένο σύνολο πόντων που πρέπει να συλλεχθούν από τους παίκτες για να νικήσουν. Οι πόντοι συλλέγονται ανάλογο με τον στόχο της κάθε λειτουργίας παιχνιδιού που επέλεξαν να ανταγωνιστούν. Οπότε, είναι στο χέρι των παικτών να ορίσουν περιορισμούς και συνθήκες νίκης εφόσον αυτοί το θελήσουν. Κάτι το οποίο δεν μπορεί να ρυθμιστεί μέσω της εφαρμογής. Το ανταγωνιστικό στοιχείο αποσκοπεί να έχει μία πιο χαλαρή και ελεύθερη διάσταση.

Inputs

Αναφορικά με τα στοιχεία εισόδου, κατά την σχεδίαση ήταν επιθυμητό το παιχνίδι να δίνει την δυνατότατα στους παίκτες να χρησιμοποιήσουν είτε πληκτρολόγιο και ποντίκι είτε ένα gamepad ανεξάρτητος μάρκας. Οπότε ήταν αναγκαίο να χρησιμοποιηθεί ο Input System. Για τον χειρισμό και τα στοιχεία εισόδου του χαρακτήρα δημιουργήθηκε ένας οδηγός χάρτης “Player”, ενώ για την κάθε κατηγορία συσκευών δημιουργήθηκε ένα control scheme (Keyboard&Mouse και Gamepad) με την κάθε συσκευή να έχει τις τα δικά της κουμπιά εισόδου για τον χειρισμό και τις ενέργειες του χαρακτήρα που προαναφέρθηκαν. Οπότε ο παίκτης ανοίγοντας το παιχνίδι, εφόσον έχει συνδεδεμένη μία τέτοια συσκευή στον υπολογιστή του, το παιχνίδι αναγνωρίζει την συσκευή και λαμβάνει την είσοδο από τα πλήκτρα. Η επιλογή των εισόδων για την καθεμία ενέργεια έγινε με βάση τα συμβατικά κουμπιά που χρησιμοποιούνται σε shooting παιχνίδια και αναγράφονται στο αρχικό μενού. Ο χειρισμός των menu και του UI πραγματοποιείται πάλι κάνοντας χρήστη του Input System ο οποίο κάνει δυνατή την αναγνώριση των συσκευών αυτών. Για την είσοδο σε αυτή την περίπτωση χρησιμοποιήθηκε ένας προεπιλεγμένος οδηγός/ χάρτης που αναφέρεται στο UI.

User Interface

Στο αρχικό μενού του παιχνιδιού πέρα από τα controls για την κάθε συσκευή υπάρχει η επιλογή “Play”, “Options” και “Quit”. Επιλέγοντας “Options” ο παίκτης είναι σε θέση να ρυθμίσει την ένταση της μουσικής και των ηχητικών εφέ. Οι ρυθμίσεις αυτές είναι προσβάσιμες και κατά την διάρκεια της μάχης, μέσω του μενού παύσης παιχνιδιού. Οι ρυθμίσεις αυτές διατηρούνται και ανανεώνονται μεταξύ των σκηνών του παιχνιδιού, ενώ παράλληλα αποθηκεύονται και κατά την έξοδο του. Επιλέγοντας “Play” ο παίκτης αμέσως έχει να επιλέξει ανάμεσα από τις δύο λειτουργείες “Practice” και “Versus” και στην συνέχεια επιλέγει πίστα. Μετά το παιχνίδι αναμένει από τον κάθε παίκτη να συμμετάσχει με την συσκευή της αρεσκείας του πατώντας οποιοδήποτε κουμπί στην συσκευή. Είναι απαραίτητο να συμμετάσχουν και οι δύο παίκτες για να μπορούν να αρχίσουν το παιχνίδι, επιλέγοντας το κουμπί “Start”. Αμέσως μετά, οι παίκτες είναι έτοιμοι και μπορούν να παίξουν. Στην περίπτωση που μία συσκευή αποσυνδεθεί, εν ώρα μάχης ή στο μενού συμμετοχής, τότε εμφανίζεται σχετικό μήνυμα. Σε περίπτωση μάχης, το παιχνίδι κάνει αυτόματη παύση και πρέπει να επανασυνδεθεί η συσκευή για να συνεχίσει η μάχη.

Σε κατάσταση μάχης, ο κάθε παίκτης στο κάτω μέρος της οθόνης του μπορεί να δει τις σφαίρες καθώς και τους γεμιστήρες του όπλου που του απομένουν, μαζί με μία πράσινη μπάρα η οποία αναπαριστά τους πόντους ζωής του. Στο επάνω και κεντρικό μέρος της οθόνης, οι παίκτες μπορούν να δουν τους πόνους που έχουν συλλέξει. Επίσης,

στο κέντρο κάθε διαχωρισμένης οθόνης ο παίκτης έχει ένα οδηγό “Crosshair”. Πατώντας παύση παιχνιδιού με την αντίστοιχη είσοδο της συσκευής, το παιχνίδι κάνει παύση ωσότου ξαναρχίσει, δίνοντάς επιπλέον την επιλογή ρυθμίσεων και επιστροφής στο αρχικό μενού του παιχνιδιού.

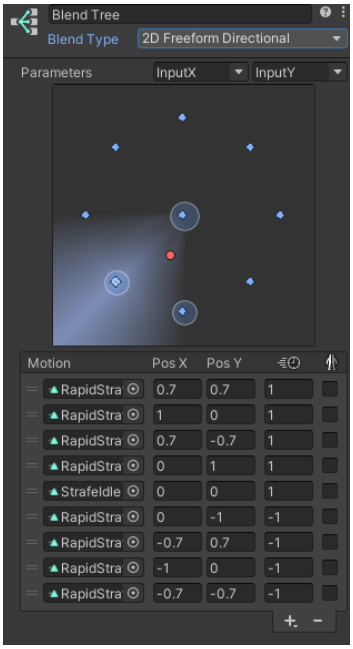
3. Υλοποίηση και Μεθοδολογία

Η διαδικασία ανάπτυξης των μηχανισμών για ένα 3rd Person Shooter αποτέλεσε μία περίπλοκη διαδικασία, επειδή ήταν αναγκαίο το αντικείμενο χαρακτήρας “Player Object Prefab” που ελέγχεται από τον παίκτη να μπορεί να επιτελέσει τις ενέργειές του αρμονικά και συγχρονισμένα σε σχέση με το αντικείμενο όπλο “Weapon” το οποίο αποτελεί μέρος του αντικειμένου χαρακτήρα. Στην παρούσα προσέγγιση προστίθεται μία επιπλέον δυσκολία, διότι αυτό το σύστημα πρέπει να λειτουργεί συγχρονισμένα για δύο τέτοιους χαρακτήρες ταυτόχρονα. Σκοπός είναι ο πειραματισμός, η κατανόηση και η διατριβή με αυτούς τους μηχανισμούς στο περιβάλλον της Unity. Για την δημιουργία αυτού του παραπάνω prefab υπήρξε αρκετή βοήθεια από παραδείγματα σε μορφή βίντεο αλλά και από συζητήσεις forums στο διαδίκτυο. Οι βάσεις για την λειτουργικότητά του χαρακτήρα αυτού επιτεύχθηκαν ακολουθώντας την προσέγγιση και μεθοδολογία αυτών των πηγών. Παρόλα αυτά, το prefab επεξεργάστηκε αρκετές φορές καθ’ όλη την διάρκεια της ανάπτυξης, τόσο στο περιβάλλον της Unity όσο και με τα σχετικά scripts που του προσφέρουν λειτουργικότητα. Με αποτέλεσμα, η λειτουργικότητα του έχει, σε μεγάλο βαθμό, διαφοροποιήσεις, αλλά και επιπλέον προσθήκες ανάλογα τους στόχους και τις ανάγκες που προέκυψαν κατά την ανάπτυξη. Μερικά παραδείγματα αυτής της προσέγγισης είναι:

- Γίνεται χρήση του Input System, προκειμένου τα scripts να λαμβάνουν την είσοδο από τις συσκευές, αυτά πρέπει αν επικοινωνούν με τις ενέργειες εισόδου “Input Actions”, αντί να λειτουργούν με συγκεκριμένα keys.
- Η εφαρμογή κάνει χρήση του διαχωρισμού οθόνης, οπότε χρειάστηκε να γίνουν επιπλέον τροποποιήσεις ώστε να εμφανίζονται σωστά τα UI στοιχεία και οι κάμερες των χαρακτήρων.
- Η ανάγκη των χαρακτήρων να εξολοθρεύονται και να πρέπει να ξανά τοποθετηθούν σωστά για τον επόμενο γύρο, επέφερε επιπλέον τροποποιήσεις.

Assets και Libraries

Τα περισσότερα αντικείμενα assets του παιχνιδιού προέρχονται από το πακέτο “POLYGON Starter Pack - Low Poly 3D Art by Synty” [8], το οποίο πρόκειται για ένα εισαγωγικό πακέτο τρισδιάστατων μοντέλων της μορφής “Low Poly”. Τα μοντέλα έρχονται με σκελετό αλλά δεν διαθέτουν animations, οπότε για τα animations του χαρακτήρα χρησιμοποιήθηκε ένα πακέτο της Unity “Standard-Assets-Characters” [13]. Τέλος, εντός την μηχανής Unity χρησιμοποιήθηκαν οι βιβλιοθήκες, Render Pipeline, Cinemachine, Animation Rigging και Input System.

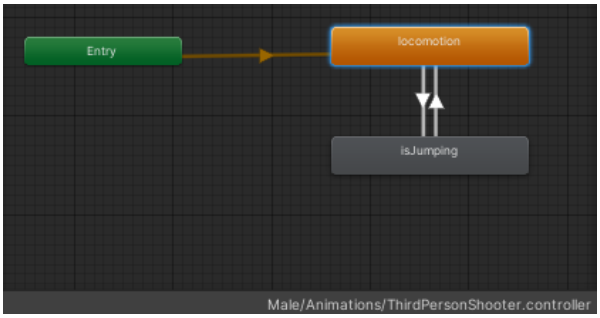


Εικόνα 1

Player Object Prefab: Locomotion και Aiming

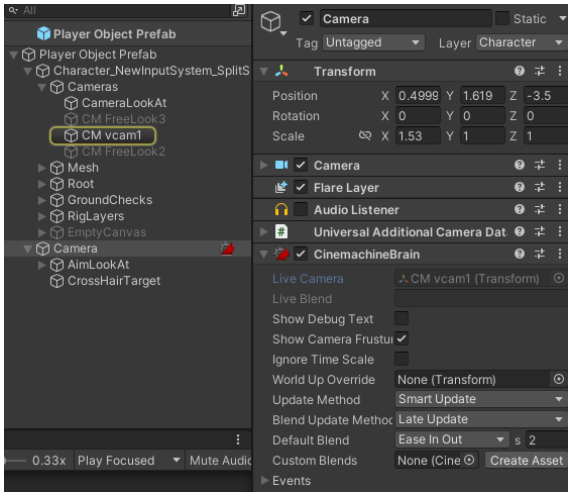
Αρχικά, δημιουργήθηκε ο “ThirdPersonShooter” animation controller [17] και χρησιμοποιώντας το πακέτο “Standard-Assets-Character” προστέθηκαν στο μοντέλο

του χαρακτήρα τα animations για την κίνηση, τα οποία ανάλογα με τις παραμέτρους κατεύθυνσης x, y αναμιγνύονται[1] (εικόνα 1). Στην συνάρτηση Update() του "CharacterLocomotion.cs" (ln 102[2]), η μεταβλητή "input" διαβάζει το διάνυσμα εισόδου για την κίνηση και επεξεργάζεται μέσω του SmoothDamp() ώστε αυτή να αλλάζει την τιμή της με πιο ομαλό τρόπο. Αμέσως, μεταφέρεται στις παραμέτρους "InputX" και "InputY" για την αναπαραγωγή των animations. Επιπλέον, γίνεται έλεγχος στην FixedUpdate() (ln 159), για το αν ο χαρακτήρας αναπηδά. Η προεπιλεγμένη τιμή οδηγεί το πρόγραμμα στο να ανανεώσει το "Character Controller" component μετακινώντας το προσθέτοντας την ταχύτητα εδάφους (ln 225). Σε συνέχεια του animation controller, εφόσον έχει δοθεί εντολή εισόδου αναπήδηση και ο χαρακτήρας βρίσκεται στο έδαφος, αναπαράγεται το αντίστοιχο animation. Οπότε, πρώτα γίνεται έλεγχος για το εάν βρίσκεται στο έδαφος (ln 143). Αυτό συμβαίνει αν μια σφαίρα μικρού μεγέθους έρχεται σε επαφή με κάποιο άλλο Layer πέρα από αυτό του χαρακτήρα[3].



Εικόνα 2

Εφόσον είναι σε θέση να αναπηδήσει προς τα πάνω (ln 191), προστίθεται στο κάθετο άξονα ταχύτητα η οποία προκύπτει από την ταχύτητα του animation και την ταχύτητα εδάφους. Αφού βρίσκεται στον αέρα (ln 161) γίνεται ανανέωση των τιμών, η κατακόρυφη ταχύτητά του μειώνετε λόγω της βαθύτητας, ενώ η οριζόντια ελέγχεται από την είσοδο κίνησης στο χώρο και μεταφέρεται τον character controller (ln 211). Οπότε με αυτόν τον τρόπο γίνονται οι εναλλαγές τον animations εντός του "ThirdPersonShooter" controller (εικόνα 2). Ένα ακόμα στοιχείο που αφορά την κίνηση είναι η οριζόντια επαφή/ σύγκρουση του character controller του χαρακτήρα με αντικείμενα τα οποία φέρουν "Rigidbody" component. Σε αυτή την περίπτωση (ln 240), τα αντικείμενα αυτά δέχονται μία δύναμη και μετακινούνται ανάλογα με την κατεύθυνσή που χτυπήθηκαν.



Εικόνα 3

Για τον κάθε χαρακτήρα χρησιμοποιείται μία "CM Virtual Camera" κάμερα η οποία αντιστοιχεί σε ένα διαφορετικό component "CinemachineBrain" (εικόνα 3). Αυτό είναι αναγκαίο προκειμένου να υπάρχει λειτουργικότητα διαχωρισμένης οθόνης [23] με βάση το Input System. Στην Update() του "CharacterAiming.cs" (ln 78),

[1] Ο τρόπος με τον οποίο διαβάζονται οι τιμές μέσω του Input System αυτές αναφέρεται παρακάτω.

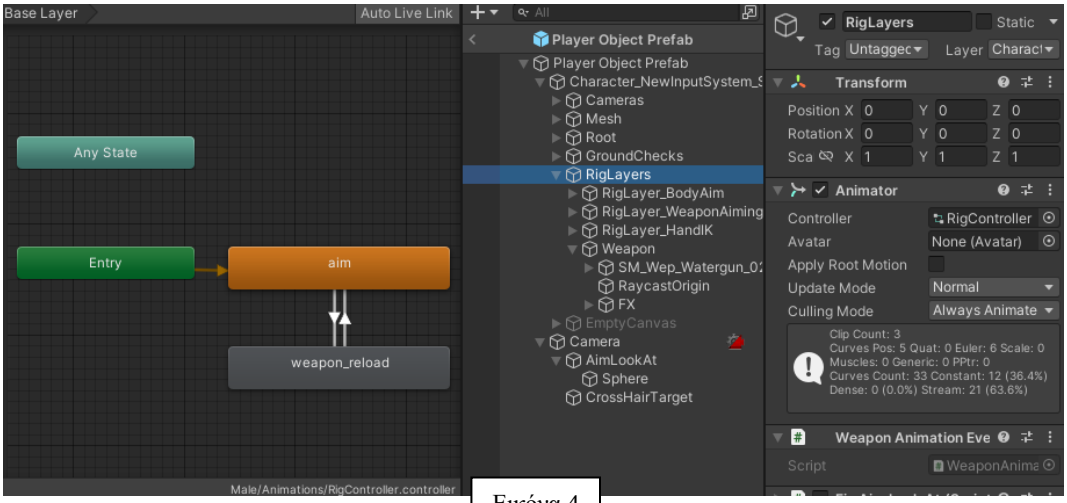
[2] Η συντομογραφία Ln αναφέρεται στην αντίστοιχη γραμμή στο έγγραφο κώδικα.

[3] Η σφαίρα αυτή δημιουργεί πρόβλημα στον έλεγχο εδάφους όταν ο χαρακτήρα βρίσκεται πολύ κοντά σε τοίχο με την πλάτη. Δηλαδή παρέμενε στο έδαφος με αποτέλεσμα να αναπηδά συνεχόμενα με αυτόν τον τρόπο. Για την αντιμετώπιση του προβλήματος γίνεται ένας επιπλέον έλεγχος (Ln 128, 137) για το εάν βρίσκεται σε τοίχο με τον τρόπο της σφαίρας και όταν αυτό ισχύει γίνεται χρονομέτρηση για το κάθε ότε μπορεί να αναπηδήσει ο χαρακτήρας.

διαβάζονται οι τιμές εισόδου για τους άξονες x, y κάθε φορά που πραγματοποιείται η ενέργεια “look”. Στο διάνυσμα αυτό προστίθεται η ταχύτητα “lookSpeed” σε συνάρτηση με τον χρόνο και περιστρέφεται το αντικείμενο “CameraLookAt” (ln 70). Σύμφωνα με την παράμετρο “Follow” της ψηφιακής κάμερας, ο κάθε χαρακτήρας είναι ρυθμισμένος να κοιτάει προς το αντικείμενο “CameraLookAt”, οπότε η οπτική ανανεώνεται με αυτόν τον τρόπο. Επιπρόσθετα (ln 72-73), περιστρέφεται και το μοντέλο προς την κατεύθυνση αυτή της κάμερας και είναι σε θέση να μετακινείται προς την κατεύθυνση αυτή. Τέλος, μέσω της εικονικής κάμερας ρυθμίζεται το offset του μοντέλου από το κέντρο.

Player Object Prefab: Animation Rigging

Για να μπορεί ο χαρακτήρας να κρατάει το όπλο και να κινείται συγχρονισμένα με αυτό σε πραγματικό χρόνο, σε αυτόν προστέθηκε ένα “Rig Builder” component μέσω της βιβλιοθήκης Animation Rigging [1] [16] [21]. Σύμφωνα με την μέθοδο αυτή, οργανώνονται σετ περιορισμών ώστε να τεθούν προϋποθέσεις σχετικά με την λειτουργία ενός σκελετού rig σε συνδυασμό με τα animations που αυτό πραγματοποιεί. Στην προκειμένη περίπτωση χρησιμοποιείται για να μπορεί ο σκελετός να «κρατάει» το όπλο του χαρακτήρα, ενώ παράλληλα είναι σε συγχρονισμό με τις κινήσεις του όπλου. Επίσης, καθώς ο χαρακτήρας στρέφεται προς την κατεύθυνση που βλέπει η κάμερα, σκοπός είναι το όπλο αλλά και όλος ο σκελετός να ακολουθούν την κίνηση αυτή[4]. Ακολουθώντας λοιπόν αυτή την μέθοδο δημιουργούνται τρία βασικά rig layers, τα οποία αφορούν την κίνηση ορισμένων τμημάτων του σκελετού σε σχέση με την κίνηση της κάμερας “RigLayer_BodyAim”, του όπλου ως προς την οπτική της κάμερας “RigLayer_WeaponAiming” και των χεριών του σκελετού που κρατάνε το όπλο και ακολουθούν την κίνησή του “RigLayer_HandIK”. Τα διαφορετικά είδη των constraints components σε καθένα από τα αντικείμενα των Rig Layers, ρυθμίζουν είτε κατά πόσο ακολουθούν/ ανταποκρίνονται συγκεκριμένα μέρη του σκελετού στην κίνηση της κάμερας
“AimLookAt” (π.χ. “AimSpine1”), είτε κατά πόσο μένουν σταθερά σε σχέση με ένα άλλο τμήμα του σκελετού ή ένα αντικείμενο (π.χ. “RightHandIK”). Στην περίπτωση των “Override Transform” [7] components, πρόκειται για ένα constraint το οποίο επιτρέπει την παράκαμψη ενός ήδη ελεγχόμενου (constrained) αντικειμένου με βάση μία πηγή παράκαμψης. Παραδείγματος χάρη, στο “BodyRecoil” η ιδιότητα transform του τμήματος σκελετού “Spine_02” ελέγχεται από το δικό του transform και αυτό γιατί υπάρχει το animation “weapon_recoil” που επηρεάζει τις τιμές του transform αυτού κάθε

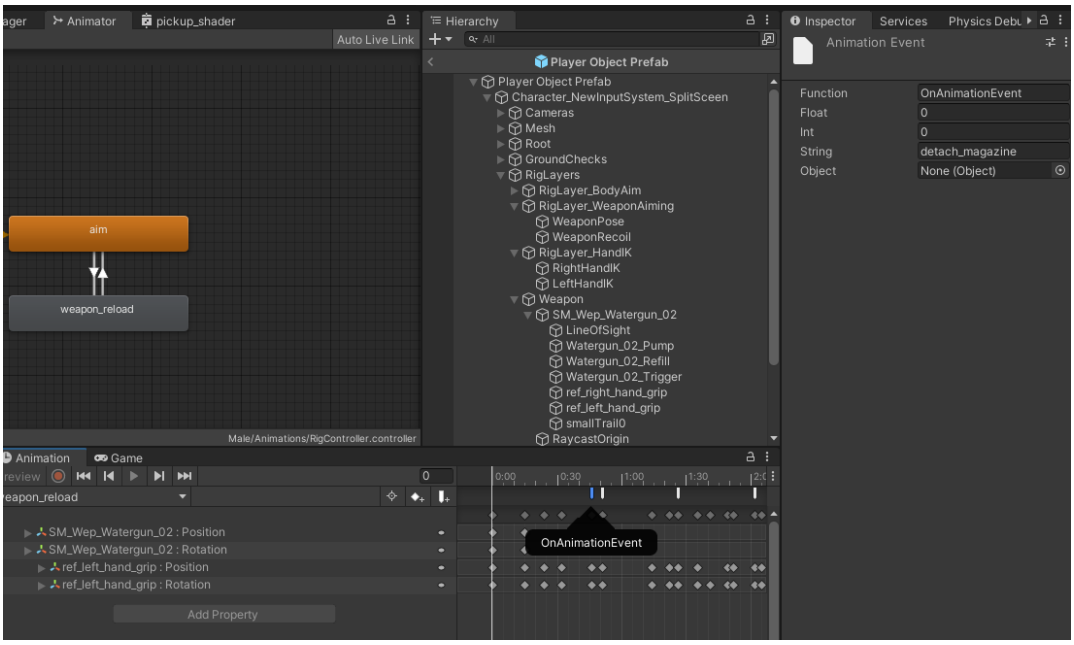


Εικόνα 4

[4] Το πόσο ακριβής είναι η κίνηση του κάθε τμήματος σκελετού σε σχέση με την οπτική της κάμερας ρυθμίζεται από την παράμετρο “Weight” του κάθε constraint.

φορά που το χαρακτήρας πυροβολεί. Τα παραπάνω rig layers εμπεριέχονται μέσα στο “RigLayers” game object, το οποίο αυτό φέρει ένα animator component συνδεδεμένο με το “RigController” controller. Με αυτό γίνεται δυνατή η μίξη μεταξύ των καταστάσεων όπου το όπλο στοχεύει “aim” και οπλίζει “weapon_reload” animations (εικόνα 4).

Η παράμετροι για την μίξη μεταξύ των καταστάσεων στο “RigController” controller ελέγχονται από το “ReloadWeapon.cs” και από το “WeaponRecoil.cs”. Στην περίπτωση που “ReloadWeapon.cs”, στην Update() γίνεται έλεγχος για το αν πραγματοποιήθηκε η ενέργεια “reload” (ln 75-79). Εφόσον μπορεί να οπλίσει, η παράμετρος “reload_weapon” γίνεται αληθής και αναπαράγεται το σχετικό animation. Στο animation υπάρχει παραμετροποίηση των τιμών transform στο μοντέλο του όπλου αλλά και στους οδηγούς που κατευθύνουν τα χέρια με την χρήση key frame. Πέρα από την κίνηση αυτών τον αντικειμένων κατά την διάρκεια του animation παρατηρείται ότι τα χέρια του χαρακτήρα αποσυνδέουν τον γεμιστήρα του όπλου και τον αντικαταστούν με έναν νέο από την «τσέπη» του. Αυτό επιτυγχάνεται χρησιμοποιώντας animation events [18] (ln 68). Ενώ αναπαράγεται το “reload_weapon” animation, καλούνται events σε μορφή συμβολοσειράς μέσω της συνάρτησης “OnAnimationEvent” του ”WeaponAnimationEvents.cs” (ln 14, εικόνα 5). Στο “ReloadWeapon.cs” υπάρχουν οι



Εικόνα 5

ενέργειες που εκτελούνται κατά αντιστοιχία των events αυτών (ln 91-138). Οπότε σύμφωνα με τις ενέργειες αυτές:

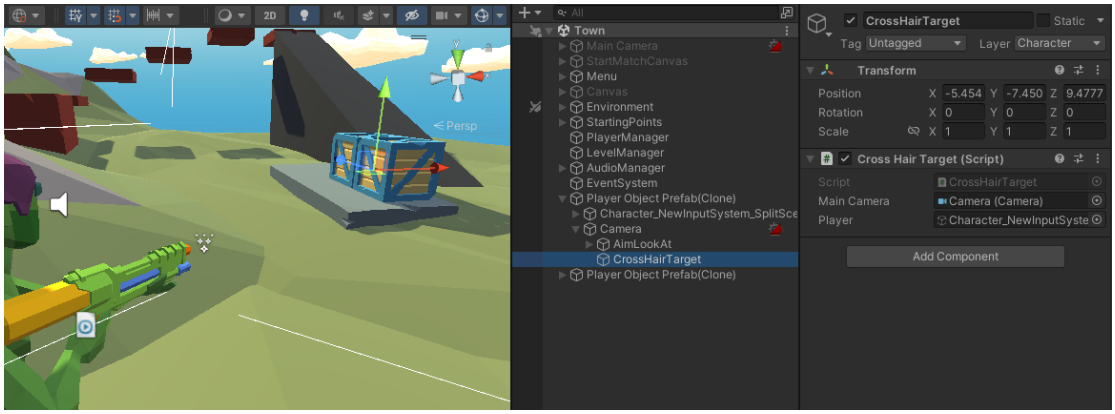
- Πρώτα αποσυνδέεται ο γεμιστήρας DetachMagazine(), χρησιμοποιώντας κάθε φορά ένα νέο αντίγραφο αυτού, ενώ το πραγματικό απενεργοποιείται προσωρινά.
- Το αντίγραφο πέφτει DropMagazine() προσθέτοντας τα components για την φυσική και αμέσως μετά εμφανίζεται ένας προσωρινός γεμιστήρας στο χέρι του την ώρα που πιάνει την «τσέπη» RefillMagazine().
- Τέλος, όταν το χέρι ξανά φτάσει στο σημείο επαφής του γεμιστήρα με το όπλο, ο προσωρινός γεμιστήρας καταστρέφεται και εμφανίζεται ξανά ο πραγματικός AttachMagazine(). Η κατάσταση μίξης στο τέλος επιστρέφει στο “aim”

Στην περίπτωση του “WeaponRecoil.cs” [19] (ln 44), κάθε φορά που ο χαρακτήρας πυροβολεί γίνεται αναπαραγωγή του “weapon_recoil” animation επηρεάζοντας τον σκελετό με τον τρόπο που περιεγράφηκε. Συμπληρωματικά η κάμερα του χαρακτήρα

δέχεται ένα ταρακούνημα με σκοπό να προσθέσει στον ρεαλισμό. Αυτό επιτυγχάνεται με το “Cinemachine Impulse Source.cs” το οποίο με την συνάρτηση GenerateImpulse() παράγει έναν κρουστικό παλμό (ln 37-42) και μετακινεί την κάμερα με βάση τον δεδομένο πίνακα διανυσμάτων “recoilPattern” (ln 52-53), εφόσον “time” ισούται με 0.2.

Player Object Prefab: Weapon

Σχετικά με την λειτουργικότητα και την λογική του όπλου, στο “CharacterAiming.cs”, στην Update() γίνονται επιπλέον έλεγχοι για το αν ο χαρακτήρας εκτελεί την ενέργεια “attack” (ln 80-89). Αυτές οι συνθήκες αναφέρονται στην ανανέωση των παραμέτρων που βρίσκονται στο game object “Weapon” του χαρακτήρα και κατ’ επέκταση στο αρχείο κώδικα “RaycastWeapon.cs” [20] [21] [22] που αυτό περιέχει. Το όπλο ξεκινάει και πυροβολεί StartFiring() “RaycastWeapon.cs” (ln 77) και επειδή “isFiring” αληθεύει στο UpdateFiring() (ln 83) αρχικά υπολογίζεται ο ρυθμός επανάληψης για την κάθε επόμενη σφαίρα που δημιουργείται και εκτοξεύεται. Δηλαδή, ο συνεχόμενος χρόνος που πραγματοποιείται η εντολή πρέπει να είναι μεγαλύτερος ή ίσος από το όριο 0.06. Στην συνάρτηση FireBullet() αρχικά ενεργοποιείται ένα “muzzleFlash” particle system στην άκρη του όπλου η οποία είναι η λάμψη[5] την στιγμή του πυροβολισμού (ln 180). Η σφαίρα δημιουργείται[6] (ln 183) με αρχικό θέση την τοποθεσία του game object “raycastOrigin” στην άκρη του όπλου, ενώ η ταχύτητά της “velocity” προέρχεται από την διαφορά της θέσης προορισμού της με την αρχική τοποθεσία. Η θέση προορισμού πηγάζει από την ακριβή τοποθεσία του “CrossHairTarget” game object που σκοπεύει ο χαρακτήρας μέσω της κάμερας (εικόνα 6). Στην συνέχεια, ενεργοποιείται το ίχνος από την διαδρομή της



Εικόνα 6

σφαίρας το οποίο πρόκειται για ένα “Trail Renderer” component στο “BulletTracer” prefab game object.

Οι σφαίρες της λίστας “bullets” ανανεώνονται συνεχόμενα μέσω της Update() του “CharacterAiming.cs”. Οπότε η συνάρτηση UpdateBullets() μεταρρυθμίζει την κατάσταση των σφαιρών (ln 92) και τις προσομοιώνει σε σχέση με τον χρόνο (ln 101). Η αρχική της θέση “p0” πάντα είναι η θέση στην οποία ξεκινά ο raycast, δηλαδή η άκρη του όπλου, ενώ η τελική της θέση “p1” έπεται από την απόσταση που έχει διανύσει την δεδομένη στιγμή που κλήθηκε η Update() κατευθυνόμενη προς τον στόχο (ln 53). Ειδικότερα, η κίνησή της πηγάζει από την παρακάτω δευτεροβάθμια εξίσωση [9]:

$$f(t) = (x0 + x*t, y0 + y*t - g*t^2/2)$$

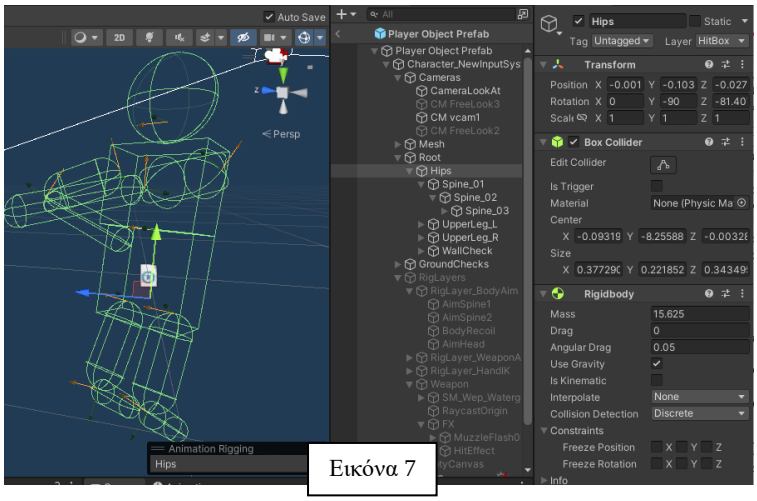
[5] Τα particle system που χρησιμοποιούνται προέρχονται από το πακέτο “Unity Particle Pack” [12].

[6] Με βάση την κλάση των σφαιρών η κάθε σφαίρα αποθηκεύει τις παραμέτρους της κατά την δημιουργία της και προστήθεται στην λίστα “bullets”.

Με αυτόν τον τρόπο ενώ η σφαίρα ταξιδεύει προς τον προορισμό, αρχίζει σταδιακά να πέφτει προς τα κάτω. Συνεχίζοντας, εξετάζεται εάν κατά την διαδρομή της η σφαίρα πέτυχε κάποιον έγκυρο στόχο RaycastSegment() (ln 106). Σε αυτήν την περίπτωση ο raycast ελέγχει αν χτύπησε κάτι αγνοώντας την μάσκα "ignorThis" layers mask. Στο σημείο που πετυχαίνει ο raycast ενεργοποιείται το δεύτερο particle system "hitEffect" που αφορά την σκόνη, τις σπίθες και το σημάδι της σφαίρας κατά την σύγκρουσή της[7]. Αν το αντικείμενο διαθέτει "Rigidbody" component, τότε αυτό δέχεται μία ώθηση (ln 128). Στην περίπτωση που πετύχει αντικείμενο που φέρει "HitBox.cs" component (ln 133) τότε το αντικείμενο αυτό πρόκειται για τον αντίπαλο παίκτη. Τότε ξεκινάει μία συνάρτηση StartCoroutine()[8] η οποία εμφανίζει για μικρό χρονικό διάστημα ένα UI εικονίδιο στο cross hair, ως ένδειξη ότι πέτυχε τον στόχο. Παράλληλα, μέσω αυτής της σύγκρουσης με το αντικείμενο hit box ο άλλος παίκτης λαμβάνει ζημιά μέσω της συνάρτησης OnRaycastHit() στο "HitBox.cs" (ln 9). Ανάλογα με tag που έχει το καθένα διαφορετικό hit box αντικείμενο, δέχεται την αντίστοιχη ζημιά, ανανεώνοντας τους πόντους ζωής στο "Health.cs" του χαρακτήρα που δέχεται την ζημιά.

Health, Death και Ragdolls

Συνεχίζοντας από την περίπτωση που ένας χαρακτήρας πυροβολήσει εύστοχα τον άλλον, ο άλλος πρόκειται να χάσει πόντους ζωής με βάση ένα σύστημα health που περιέχει στο prefab του. Το σύστημα αυτό λειτουργεί μέσω του "Health.cs". Αρχικά, όταν δημιουργείται ο χαρακτήρας η πόντου ζωής του είναι ίσοι με τους μέγιστους (ln 38), ενώ στην Update() η πόντοι αυτοί ανανεώνονται και στον UI της μπάρας ζωής "healthBar" (ln 50). Κάθε φορά που καλείται η συνάρτηση TakeDamage() (ln59), από τους πόντους αυτούς μειώνεται ένα μέρος ανάλογα με το hit box tag όπως προαναφέρθηκε. Στην περίπτωση που ο χαρακτήρας δεν έχει άλλους πόντους ζωής βρίσκεται προσωρινά σε κατάσταση «θανάτου»[9]. Τότε ο άλλος παίκτης αποκτάει ένα πόντο με αυτό να ανανεώνεται και στο UI του στο κέντρο της οθόνης και από την δική του μεριά της μισής οθόνης (ln 97). Επιπλέον, ξεκινάει μία συνάρτηση StartCoroutine(), με σκοπό μετά από το διάστημα τριών δευτερολέπτων να γίνουν οι προετοιμασίες και οι ρυθμίσεις για τον επόμενο γύρο (ln 103). Οι ρυθμίσεις αυτές πραγματοποιούνται στο "LevelMangaer.cs"[10] και θα αναφερθούν έπειτα παρακάτω.



Εικόνα 7

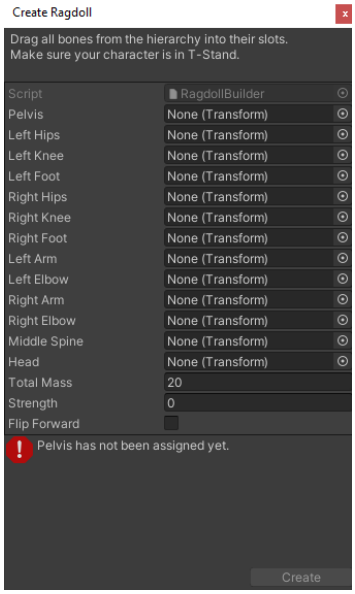
Στον κάθε χαρακτήρα έχει προστεθεί ένα σύστημα Ragdoll [2], το οποίο πραγματοποιεί δύο λειτουργίες. Η πρώτη αφορά την προσθήκη επιμέρους hit boxes/ box colliders στα τμήματα του σκελετού του, με αποτέλεσμα να είναι πιο ακριβή τα όρια με τα οποία μπορούν να τον

[7] Η σφαίρα καταστρέφεται από την λίστα έπειτα από τρία δευτερόλεπτα (ln 98).

[8] Οι συναρτήσεις αυτές επιτρέπουν την προσθήκη καθυστέρησης delay στις ενέργειες που πρόκειται να εκτελέσουν.

[9] Ήταν σημαντικό να απενεργοποιηθεί η μεταβλητή "isFiring" και το σχετικό εικονίδιο πριν την έναρξη του νέου γύρου, διότι διαφορετικά θα συνέχιζε να πυροβολεί στην περίπτωση που ήδη πυροβολούσε αμέσως πριν «πεθάνει» (ln 68-69.

[10] Το όνομα γράφτηκε εσφαλμένα με αυτόν τον τρόπο, το σωστό θα ήταν "Levelmanager".



Εικόνα 8

πυροβολήσουν (εικόνα 7). Η δεύτερη λειτουργία αφορά την χρήση του κάθε "RigidBody" component στα hit boxes, ώστε όταν ο χαρακτήρας εξοντωθεί να εφαρμοστεί στο μοντέλο φυσική που το καθιστά να καταρρέει. Για την δημιουργία του ragdoll συμπληρώνονται τα ζητούμενα τμήματα του σκελετού του μοντέλου και αμέσως δημιουργούνται σε καθένα τμήμα τα "Box Collider", "Rigidbody" και "Character Joint" components τα οποία είναι απαραίτητα για τις παραπάνω λειτουργίες. Για την λειτουργία των hit boxes σε κάθε ένα από τα rigid bodies στον χαρακτήρα προστίθεται ένα "HitBox.cs" component[11] και όταν αυτά χτυπηθούν από το raycast του όπλου ο χαρακτήρας δέχεται ζημία. Για την δεύτερη λειτουργία, ο κάθε χαρακτήρας έχει ένα "Ragdoll.cs" component στο οποίο μέσω του "LevelManager.cs" ενεργοποιεί και απενεργοποιεί την λειτουργία κατάρρευσης του μοντέλου[12].

Κατά την ενεργοποίηση του ActivateRagdoll() (ln 42), εάν ο χαρακτήρας είναι σε κατάσταση θανάτου, για κάθε ένα "RigidBody" component απενεργοποιείται η παράμετρος "isKinematic" ώστε αυτά να ελέγχονται από τα animations αλλά και από την φυσική των "RigidBody" components. Επίσης, απενεργοποιούνται και τα υπόλοιπα συστήματα και components τα οποία του οφείλονται για τον χειρισμό του χαρακτήρα. Δηλαδή ο χειρισμός για την κάμερα, τα animations και την κίνηση. Στην περίπτωση που πρόκειται για τον χαρακτήρα που δεν βρίσκεται σε κατάσταση θανάτου, τότε απενεργοποιούνται μόνο ο χειρισμός του χαρακτήρα και της κάμερα. Η απενεργοποίηση DeactivateRagdoll() (ln 32), για κάθε ένα "RigidBody" component ενεργοποιείται η παράμετρος "isKinematic" ώστε αυτά να ελέγχονται μόνο από τα animations του σκελετού. Φυσικά επαναφέρονται και τα προεπιλεγμένα συστήματα χειρισμού.

Pick Ups

Στο παιχνίδι παρατηρούνται δύο είδη αντικειμένων που μπορεί να συλλέξει ο παίκτης. Αυτά αφορούν την συλλογή γεμιστήρων για το όπλο και την αναπλήρωση ζωής. Η λειτουργικότητά τους πραγματοποιείται στα "AmmoPickup.cs" και "HealthPickup.cs" κατ' αντιστοιχία. Η λειτουργία τους είναι αρκετά παρόμοια και ουσιαστικά όταν αυτά έρθουν σε επαφή[13] με ένα άλλο "Collider" component, ελέγχουν αν το αντικείμενο που ήρθαν σε επαφή διαθέτει "RaycastWeapon.cs" component (ln 22) ή "Health.cs" component (ln 23). Εκτελούν δύο διαφορετικές λειτουργίες αλλά έχουν ως κοινή λειτουργία την συνάρτηση DisablePickUp(). Η συνάρτηση απενεργοποιεί προσωρινά την λειτουργία τους να τα συλλέξουν οι χαρακτήρες, με την κάθε κατηγορία να έχει διαφορετικό χρονικό διάστημα που είναι απενεργοποιημένη. Η διαδικασία απενεργοποίησης πραγματοποιείτε για άλλη μια φορά με συνάρτηση της μορφής StartCoroutine(). Για παράδειγμα, στην περίπτωση του "AmmoPickUp.cs" (ln 35) αρχικά

[11] Η προσθήκη αυτών πραγματοποιείται στο "Health.cs" (ln 40-45).

[12] Η λειτουργία καλείτε όταν ένας από τους χαρακτήρες βρίσκεται σε κατάσταση θανάτου "LevelManager.cs" (ln 45).

[13] Η λειτουργία "Is Trigger" στο "Sphere Collider" component είναι ενεργοποιημένη ώστε να μπορεί να γίνει αυτός ο έλεγχος μέσω του κώδικα.

το αντικείμενο αλλάζει σε κόκκινο υλικό για να παρουσιάζεται ως απενεργοποιημένο, παράλληλα απενεργοποιείτε “active = false”. Αφού περάσει το χρονικό διάστημα των τριάντα δευτερολέπτων γίνεται ξανά ενεργό και λαμβάνει πάλι το αρχικό του υλικό (ln 42-47).

Level Manager και Main Camera

Όταν οι παίκτες έχουν φτάσει στο σημείο πριν ξεκινήσει η πίστα η κάμερα παρουσιάζει τον χώρο ως μία κοινή κάμερα. Πρόκειται για το “Main Camera” game object το οποίο κάνει δυνατή αυτή την λειτουργία. Η κάμερα περιέχει το “ToggleOnPlayerJoin.cs” component το οποίο, μεταξύ άλλον, επιτελεί κάποιες σημαντικές λειτουργιές την στιγμή που και οι δύο παίκτες κάνουν “join” και ξεκινήσουν το παιχνίδι μέσω της επιλογής “Start”[14] StartMatch() (ln 87). Το “AudioListener” component από προεπιλογή βρίσκεται στο αντικείμενο αυτή της κύριας κάμερας, αλλά στην συνάρτηση ενεργοποιείται Το “AudioListener” component της κάμερα του πρώτου χαρακτήρα (ln 106). Έπειτα, ενεργοποιεί και απενεργοποιεί τα απαραίτητα UI στοιχεία και τέλος απενεργοποιεί το ίδιο το game object μαζί και με το “AudioListener” component του (ln 117-128).

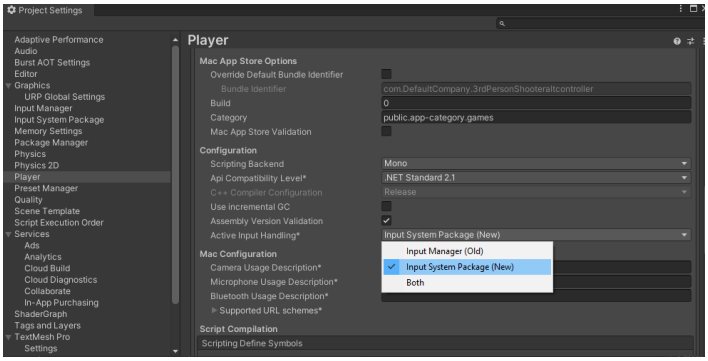
Αναφορικά με το “LevelMangaer.cs” component του“LevelManager” game object, αυτό επιτελεί μερικές από τις λειτουργίες που προαναφέρθηκαν αλλά και άλλες σχετικά με την εξέλιξη και την ομαλή ποριά της κάθε πίστας. Οι λειτουργίες αυτές χωρίζονται σε λειτουργίες που αφορούν τα στοιχεία UI, το Input System και του level/ πίστα.

Μια από τις βασικές λειτουργίες είναι η προετοιμασία για τον επόμενο γύρο (ln 186)[15]. Καλείτε η συνάρτηση RespawnPlayers() (ln 213) και τοποθετεί τους χαρακτήρες στις αρχικές τους θέσεις, επαναφέρει τους πόντους ζωής τους στους μέγιστους, τους γεμιστήρες και τις σφαίρες των όπλων στις αρχικές τους τιμές και απενεργοποιεί τα ragdolls. Αμέσως, γίνεται και η επαναφορά όλων των Pick ups αντικειμένων που συλλέγει ο χαρακτήρας (ln193). Σε καθένα από αυτά, καλείτε η συνάρτηση ResetPickUp(), σταματώντας όλες τις “Coroutines” ενέργειες και επαναφέροντας τις αρχικές ρυθμίσεις.

Πέρα από αυτές τις ενέργειας, στο “LevelMangaer.cs” ρυθμίζονται και οι καταστάσεις στις οποίες βρίσκονται τα διαφορετικά menus κατά την διάρκεια της πίστας. Μερικές από τις λειτουργίες αυτές καλούνται είτε με την επιλογή κάποιου UI στοιχείου όπως ένα κουμπί, είτε επιτελώντας μια ενέργεια μέσω του Input System, ή ακόμη και από την αλλαγή κατάσταση που μπορεί να βρίσκεται μία συσκευή εισόδου. Οι περιπτώσεις αυτές περιγράφονται στην συνέχεια.

Input System, Player Manager και Split Screen

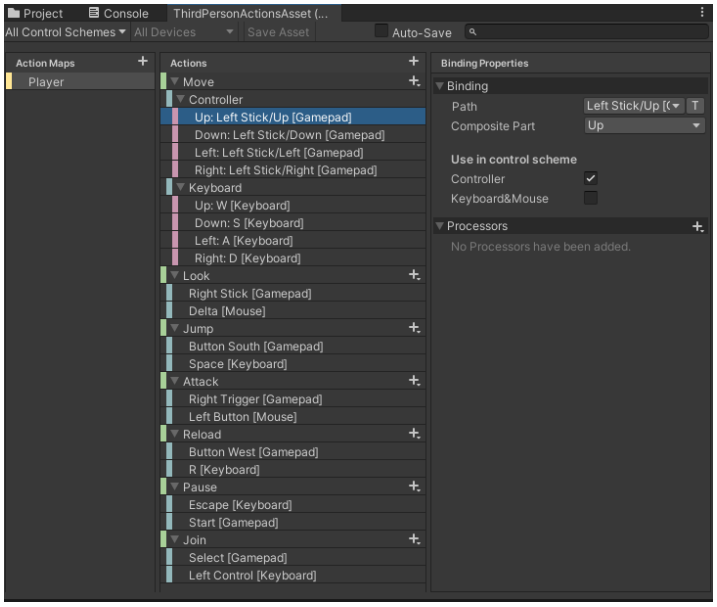
Για την χρήση του Input System [6] πρώτα προστέθηκε η σχετική βιβλιοθήκη από το την καρτέλα “Package Manager” και έπειτα στις ρυθμίσεις του project επιλέχθηκε για τον χειρισμό των εισόδων (εικόνα 9).



Εικόνα 9

[14] Η συνάρτηση καλείτε μέσω UI OnClick() event της επιλογής “Start”. Επίσης, η συνάρτηση χρησιμοποιεί παραμέτρους από άλλα αρχεία κώδικα τα οποία θα αναφερθούν στην συνέχεια.

[15] Η μεταβλητή “scored” χρησιμοποιείται ώστε να μετρηθεί μία και μόνο φορά το σκορ από την εξόντωση του άλλου χαρακτήρα. Αυτή επαναφέρεται με την έναρξη του νέου γύρου.

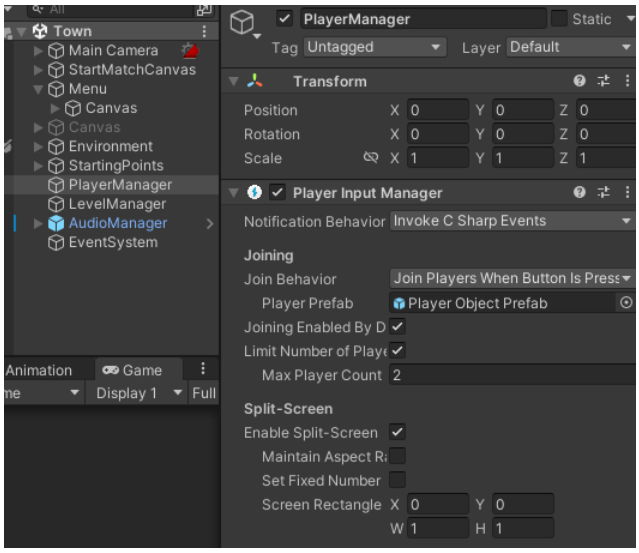


Εικόνα 10

Έπειτα δημιουργήθηκε το "ThirdPersonActionsAsset" action asset (εικόνα 10). Αυτό συγκροτεί όλες τις ενέργειες εισόδου που μπορεί να πραγματοποιήσει ο παίκτης, μαζί με τα αντίστοιχα κουμπιά (bindings) των ενεργειών για την κάθε συσκευή. Τα κουμπιά αυτά χωρίζονται σε δύο "Control Schemes" συστήματα, το "Keyboard&Mouse" και το "Controller". Στο παράθυρο αυτό δημιουργήθηκε το σύστημα χειρισμού για αυτές τις συσκευές θέτοντας κάθε φορά τα επιθυμητά κουμπιά[16].

Αξίζει να σημειωθεί ότι στην περίπτωση του "Controller" control scheme, αυτό δεν περιορίζεται σε ένα συγκεκριμένο είδος η μοντέλο gamepad, αλλά κάνει χρήση όλων των gamepads. Οπότε οι παράμετροι bindings ρυθμίστηκαν με βάση απόλυτων κατευθύνσεων που καλύπτει όλο το εύρος των κοινών gamepads.

Για να μπορούν οι παίκτες να ελέγχουν τον δικό τους χαρακτήρα, είναι αναγκαίο να υπάρχει ένα "Player Input Manager" component στην σκηνή. Οπότε αυτό προστέθηκε στο "PlayerManager" game object. Στο "Player Input Manager" component, η ενημέρωση του κώδικα παιχνιδιού για αλλαγές που συμβαίνουν αναφορικά με τους παίκτες, γίνεται μέσω συμβάντων εντός του κώδικα[17]. Η συνθήκη εισόδου νέου παίκτη είναι ενεργοποιημένη από προεπιλογή και ρυθμίστηκε ώστε η είσοδος να πραγματοποιείται με το πάτημα ενός κουμπιού. Επιπλέον, στο πεδίο "Player Prefab" επιλέχθηκε το "Player Object Prefab", το οποίο αφορά το αντίγραφο game object που θα καλεί το Input System με το πάτημα ενός κουμπιού. Τέλος, τέθηκε όριο δύο παικτών και ενεργοποιήθηκε ο διαχωρισμός οθόνης με κατακόρυφο τρόπο (εικόνα 11).



Εικόνα 11

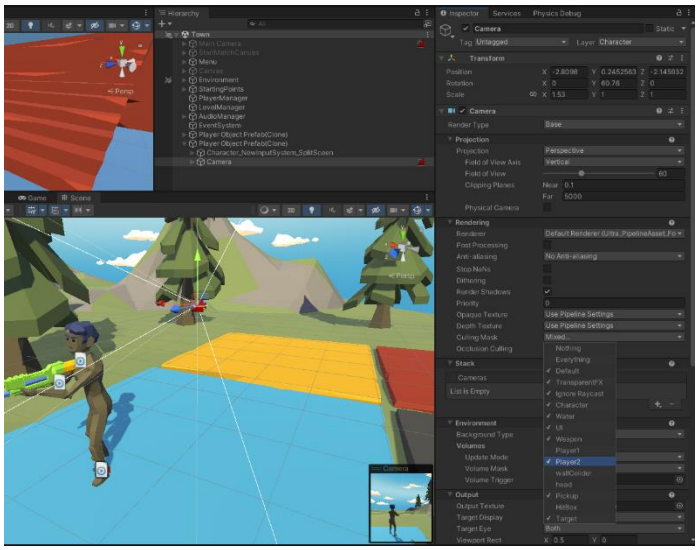
[16] Όλα αυτά αναφέρονται στο ίδιο "Player" action map το οποίο είναι και αυτό που χρησιμοποιούν οι χαρακτήρες για να πραγματοποιήσουν όλες τις ενέργειες με βάση τα προαναφερθέντα.

[17] Σύμφωνα δηλαδή με την συμπεριφορά "Invoke CSharp Events" [5]. Με τον τρόπο αυτό μπορούν να προγραμματιστούν λειτουργείες της μορφής συμβάντων (events). Για παράδειγμα, την στιγμή που ένας παίκτης εισέλθει (join) στο παιχνίδι κ.α.

Επίσης, προστέθηκε το “Player Input” component στο “Player Object Prefab” και στο πεδίο “Actions” επιλέχθηκε το “ThirdPersonActionsAsset”, ενώ στο πεδίο “Camera” επιλέχθηκε η κάμερα μέσα στο prefab. Η κάμερα αυτή είναι η κάμερα που δείχνει στο κάθε χαρακτήρα ξεχωριστά κατά τον διαχωρισμό οθόνης.

Για να μπορεί ο κάθε χαρακτήρας να ελέγχεται από μία διαφορετική συσκευή χρειάζεται το “ThirdPersonActionsAsset” input action asset να αναφέρεται ξεχωριστά για τον κάθε χαρακτήρα. Το οποίο επιτυγχάνεται μέσω του “PlayerInput” component. Άρα κάθε φορά που ο παίκτης εκτελεί μέσω της συσκευής μία ενέργεια, τότε αυτή η ενέργεια καλείτε μέσω αυτού του εκάστοτε input action asset. Συγκεκριμένα, στο “CharacterLocomotion.cs” (ln 60) οι είσοδοι “inputAsset” αναφέρονται στις ενέργειες αυτού του “PlayerInput” conponent που φέρει το αντικείμενο/ κλώνος του “Player Object Prefab”. Στην συνέχει ο οδηγός (map) για τα «κουμπιά» προέρχεται από το “Player” action map που προαναφέρθηκε[18]. Τέλος, χρειάζεται να αναφερθεί η κάθε διαφορετική ενέργεια που χρησιμοποιείται μέσω του αρχείου κώδικα. Αυτό επιτυγχάνεται κάνοντας αναφορά σε κάθε μία από αυτές με την χρήση συμβολοσειρών (ln 81-94)[19]. Οπότε, οι υπόλοιπες ενέργειες σε άλλα κομμάτια κώδικα πραγματοποιούνται με αντίστοιχο τρόπο.

Επειδή το “Player Input Manager” component επιτρέπει την κλήση συμβάντων μέσω κώδικα, μπορούν να παραχθούν ενέργειες κατά την είσοδο ενός παίκτη. Στο “PlayerManager.cs”[20] (ln 28) κάθε φορά που ένας παίκτης συνδέετε καλείτε η συνάρτηση AddPlayer(). Εκεί, οι είσοδοι των παικτών προστίθενται στην λίστα “players” (ln 38) η οποία έχει και την λειτουργία του μετρητή των παικτών[21] στην σκηνή. Κατά την σύνδεση το αντίγραφο του “Player Object Prefab”, ανάλογα την θέση του στην λίστα “players”, γεννάται στην τοποθεσία των αντικειμένων στην λίστα “startingLocations” (ln 45-47). Στην συνέχει για να έχει ο καθένας την δική του οπτική, πρέπει κάθε εικονική “CinemacgineVirtualCamera” κάμερα να επικοινωνεί με ένα ξεχωριστό “CinemachineBrain” component. Αυτό πραγματοποιείται αλλάζοντας την παράμετρο “Culling Mask” [15] του “CinemachineBrain” component, προσθέτοντας σε αυτή το αντίστοιχο “Layer” που βρίσκεται η ψηφιακή κάμερα στο αντίγραφο prefab (ln 50-55, εικόνες 12 και 13).



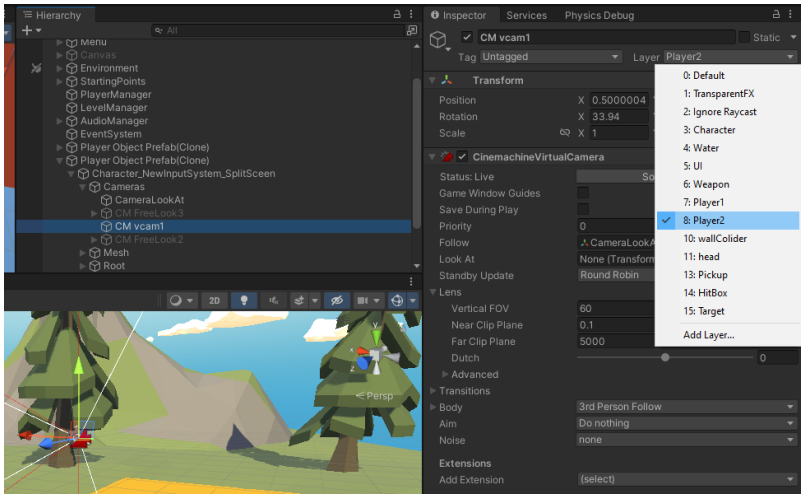
Εικόνα 12

[18] Στο σημείο αυτό γίνεται και αναγνώριση της συσκευής οπότε ο χαρακτήρας είναι δεσμευμένος με την συσκευή αυτή και χρησιμοποιεί το αντίστοιχο control scheme.

[19] Οι συναρτήσεις OnEnable() και OnDisable() πρόκεινται για όταν το αντικείμενο είναι ενεργό ή αντίστοιχα ανενεργό. Όταν ενεργοποιούνται γίνεται εγγραφή στις ενέργειες που αναφέρονται., ενώ όταν απενεργοποιείται διαγράφονται.

[20] Είναι σημαντικό να αναφερθεί πως κατά την είσοδο στην σκηνή η προεπιλεγμένη παράμετρος της ταχύτητας του χρόνου βρίσκεται στο 0 οπότε η σκηνή είναι σε παύση μέρη αυτό να γίνει 1 κατά την έναρξη.

[21] Κάποια κομμάτια κώδικα εκμεταλλεύονται και χρησιμοποιούν τον αριθμό αυτόν των παικτών για να θέσουν προκειμένου να καλέσουν άλλες ενέργειες.

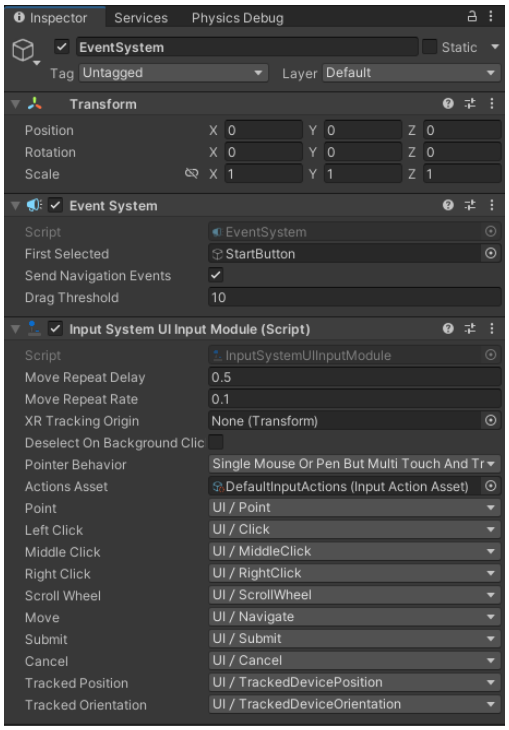


Εικόνα 13

Μία άλλη λειτουργία του Input System είναι ο έλεγχος κατάστασης των συσκευών [4]. Για παράδειγμα, στο "ToggleOnPlayerJoin.cs" (ln 36-68) με την ενέργεια "onDeviceChange" του System Input, την ώρα που αποσυνδέετε μία συσκευή εμφανίζεται για τη συσκευή αυτή σχετικό μήνυμα αποσύνδεσης. Αντίθετα όταν επανασυνδέεται εμφανίζεται μήνυμα ότι η συσκευή είναι συνδεδεμένη.

UI, Menus, Canvas και Event System

Τα διαφορετικά μενού σε όλες τις σκηνές λειτουργούν με OnClick() events επιλέγοντας τα σχετικά κουμπιά στο UI. Επιλέγοντάς για παράδειγμα το "Resume" κουμπί όταν το παιχνίδι βρίσκεται σε κατάσταση παύσης "LevelManager.cs" (ln 127-150), το παιχνίδι επαναφέρεται από την κατάσταση παύσης. Αυτό συμβαίνει εφόσον γίνουν οι απαραίτητοι έλεγχοι για το αν και οι δύο παίκτες έχουν συνδεδεμένη συσκευή για να ελέγξουν τον χαρακτήρα τους (ln 133). Τότε, για τον κάθε χαρακτήρα ενεργοποιείται το "CharacterLocomotion.cs" για να μπορεί ξανά να κινηθεί, η ταχύτητα χρόνου επαναφέρεται στην κανονική και το μενού "pauseMenu" στο τέλος απενεργοποιείται.

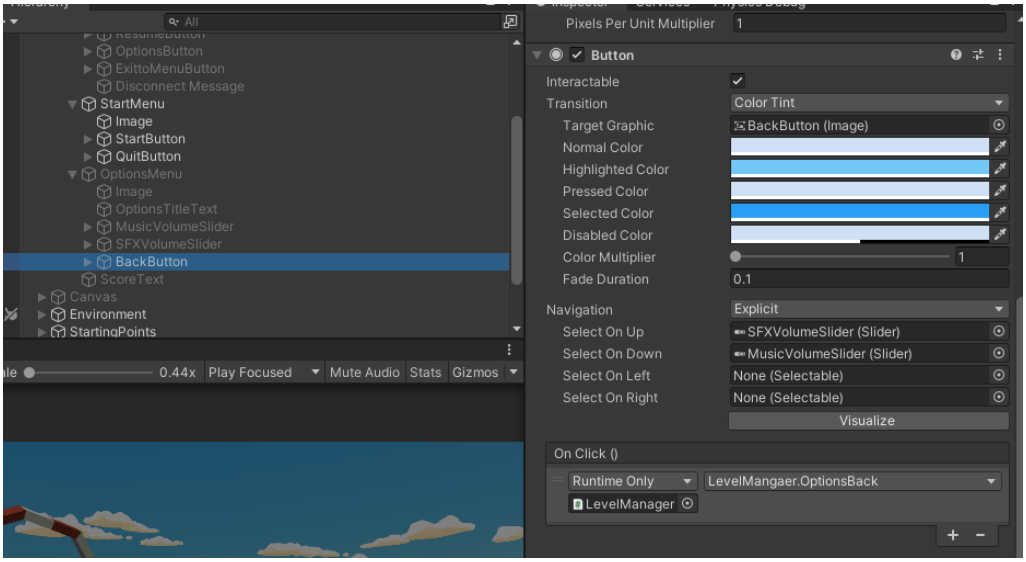


Εικόνα 14

Είναι σημαντικό να αναφερθεί πως γίνεται η πλοήγηση [11] ανάμεσα στις επιλογές αυτές του UI. Αρχικά, στο "EventSystem" game object υπάρχουν τα "Event System" και "Input System UI Input Module" components (εικόνα 14).
Η κύρια λειτουργία για το δεύτερο component αφορά την είσοδο από τις συσκευές η οποία ελέγχει τα στοιχεία του UI.

Για τον σκοπό αυτό χρησιμοποιείται το προεπιλεγμένο "DefaultInputActions" action asset του Input System που αφορά τον χειρισμό του UI. Αυτό διαθέτει τα δικά του controls schemes για πληκτρολόγιο/ ποντίκι και gamepad στο UI action map, οπότε κάνει

δυνατή την χρήση και των δύο για τον έλεγχο του UI με ευρέως χρησιμοποιημένα bindings. Το “Event System” component με την παράμετρο “First Selected” γίνεται η προεπιλογή του πρώτου UI στοιχείου κατά την περιήγηση του παίκτη μεταξύ των menus. Η παράμετρος αυτή, προτού της τεθεί ένα νέο στοιχείο χρειάζεται αν διαγραφεί το προηγούμενο. Για παράδειγμα “LevelManager.cs” (ln 167-168), όταν επιλέγεται το κουμπί “Back” για να επιστραφεί ο παίκτης από το “Options Menu” στο “Pause Menu”, το τρέχον προεπιλεγμένο “First Selected” στοιχείο αντικαθίσταται από το “pauseFirstButton” το οποίο είναι η επιλογή “Resume”. Επιπρόσθετα, η πλοήγηση ανάμεσα στα στοιχεία αυτά, επιτελείται σε συνδυασμό με την παράμετρο “Navigation” του κάθε κουμπιού το οποίο ορίζει πως θα μπορεί να κινηθεί ο παίκτης στο περιβάλλον του UI. Στο ίδιο παράδειγμα με το κουμπί “Back” του “Options Menu”, η πλοήγηση γίνεται μεταξύ αυτού και των δύο sliders για την ρύθμιση του ήχου (εικόνα 15).



Εικόνα 15

Audio Manager

Ο ήχος[22] [10] στις σκηνές αναπαράγεται μέσω του “Audio Listener” component το οποίο βρίσκεται στις κάμερες. Παρόλα αυτά, επειδή πρέπει να υπάρχει μόνο ένα τέτοιο component ενεργό στην σκηνή, πριν την έναρξη του παιχνιδιού ο ήχος αναπαράγεται μέσω της “Main Camera”, ενώ κατά την έναρξη αυτός ο “Audio Listener” απενεργοποιείται και αντικαθίσταται από τον “Audio Listener” της κάμερας του πρώτου παίκτη “ToggleOnPlayerJoin.cs” (ln 107-111).

Οι ήχοι αυτοί χωρίζονται σε δύο κατηγορίες: “Music” και “Sound Effects”. Αυτά ρυθμίζονται από τους αντίστοιχους sliders στο “Options Menu”. Όπως αναφέρθηκε ποιούμενος, οι sliders αυτοί αποτελούν κουμπιά του UI και όταν αλλάζει η τιμή τους, OnValueChanged(), καλούν μία συνάρτηση. Το “AudioManager.cs” component του “AudioManager” game object, είναι υπεύθυνο για τις λειτουργίες του ήχου. Επίσης, το αντικείμενο αυτό περιέχει το “Audio Source” component για την μουσική, ενώ το θυγατρικό αντικείμενο μέσα σε αυτό “SFX” περιέχει το “Audio Source” για την αναπαραγωγή των ηχητικών εφέ.

Στο “AudioManager.cs” (ln 66-70), η συνάρτηση PlaySFX() καλείτε από διάφορα κομμάτια κώδικα με σκοπό να αναπαράγει το ηχητικό εφέ που παίρνει ως είσοδο. Αυτό το κάνει για μία φορά μέχρι να τελειώσει το δείγμα με την συνάρτηση “PlayOneShot()”.

[22] Οι ήχοι που προστέθηκαν για την υλοποίηση του παιχνιδιού προήλθαν από την διαδικτυακή βιβλιοθήκη δειγμάτων ήχου “Zapsplat” [3]. Αυτά, αλλά και η μουσική [14] πρόκεινται για δωρεάν και royalty free δείγματα.

Συνεχίζοντας, χρησιμοποιώντας στατικές μεταβλητές της μορφής player preferences (ln 7-9), γίνεται δυνατή η αποθήκευση της έντασης της μουσική αλλά και για των ηχητικών εφέ. Επίσης, υπάρχει και η μεταβλητή "FirstPlay" για να ορίζονται κάποιες προεπιλεγμένες τιμές την πρώτη φορά που ανοίγεται η εφαρμογή. Σε άλλη περίπτωση οι πληροφορίες αυτές αντλούνται από τις αποθηκευμένες τιμές των μεταβλητών player preferences (ln 35-38). Με αυτό το σύστημα κάθε φορά που ρυθμίζεται στον slider η στάθμη του ήχου καλείτε η αντίστοιχη συνάρτηση και ανανεώνει/ αποθηκεύει την τιμή (ln 42 και 48).

4. Συμπεράσματα, αξιολόγηση και μελλοντικές επεκτάσεις

Η εργασία αποσκοπούσε στον πειραματισμό και στην μελέτη των παραπάνω λειτουργιών για την δημιουργία ενός ανταγωνιστικού παιχνιδιού σκοποβολής τρίτου προσώπου για δύο άτομα. Το αποτέλεσμα δεν είναι αρκετά ικανοποιητικό και ευχάριστο σε βαθμό που κάποιος θα ασχολούταν με αυτό για αρκετές ώρες, αλλά ίσως αποδίδει μία ικανοποίηση για μικρό χρονικό διάστημα εάν τα άτομα που παίζουν ευχαριστιούνται τον τρόπο παιξίματος. Σχετικά με τους μηχανισμούς και τον τρόπο παιξίματος, αυτά θα μπορούσαν να είναι περισσότερο βάθος και λεπτομέρεια. Ως μελλοντική επέκταση για παράδειγμα θα ήταν ενδιαφέρουσα η προσθήκη επιπλέον όπλων και εργαλείων, διαφοροποίηση ως προς του χαρακτήρες και ποικιλία στον τρόπο παιξίματος, game modes. Παρόλα αυτά, η μελέτη των μεθόδων που ακολουθήθηκαν στην ανάπτυξη αυτής της εργασίας θα βοηθήσει σίγουρα στην ανάπτυξη ενός μελλοντικού βελτιστοποιημένου παιχνιδιού.

5. Αναφορές

[1] Animation rigging: Animation rigging: 0.2.7-preview. Animation Rigging | 0.2.7-preview. (n.d.). Retrieved January 29, 2023, from https://docs.unity3d.com/Packages/com.unity.animation.rigging@0.2/manual/index.html

[2] Creating ragdolls - 2019. Unity Learn. (n.d.). Retrieved January 29, 2023, from https://learn.unity.com/tutorial/creating-ragdolls-2019#

[3] Download free sound effects at zapsplat. ZapSplat. (2023, January 20). Retrieved January 29, 2023, from https://www.zapsplat.com/

[4] Enum InputDeviceChange: Input system: 1.0.2. Input System | 1.0.2. (n.d.). Retrieved January 29, 2023, from https://docs.unity3d.com/Packages/com.unity.inputsystem@1.0/api/UnityEngine.InputSystem.InputDeviceChange.html

[5] GameObject components for input: Input system: 1.0.2. Input System | 1.0.2. (n.d.). Retrieved January 29, 2023, from https://docs.unity3d.com/Packages/com.unity.inputsystem@1.0/manual/Components.html#notification-behaviors

[6] Input system: Input system: 1.1.1. Input System | 1.1.1. (n.d.). Retrieved January 29, 2023, from https://docs.unity3d.com/Packages/com.unity.inputsystem@1.1/manual/index.html

[7] Override transform: Animation rigging: 1.1.1. Animation Rigging | 1.1.1. (n.d.). Retrieved January 29, 2023, from https://docs.unity3d.com/Packages/com.unity.animation.rigging@1.1/manual/constraints/OverrideTransform.html

[8] Polygon starter pack - low poly 3D art by Synty: 3D props. Unity Asset Store. (n.d.). Retrieved January 29, 2023, from https://assetstore.unity.com/packages/3d/props/polygon-starter-pack-low-poly-3d-art-by-synty-

156819?aid=1011ljjCh&utm_campaign=unity_affiliate&utm_medium=affiliate&utm_source=partnerize-linkmaker
[9] Schatzeder, D. (2020, December 16). Basic trajectory prediction in Unity. Medium. Retrieved January 29, 2023, from https://schatzeder.medium.com/basic-trajectory-prediction-in-unity-8537b52e1b34
[10]Technologies, U. (n.d.). Audio. Unity. Retrieved January 29, 2023, from https://docs.unity3d.com/Manual/Audio.html
[11]UI support: Input system: 1.0.2. Input System | 1.0.2. (n.d.). Retrieved January 29, 2023, from https://docs.unity3d.com/Packages/com.unity.inputsystem@1.0/manual/UISupport.html
[12]Unity Particle Pack: Tutorial Projects. Unity Asset Store. (n.d.). Retrieved January 29, 2023, from https://assetstore.unity.com/packages/essentials/tutorial-projects/unity-particle-pack-127325
[13]Unity-Technologies. (n.d.). Standard-assets-characters/assets/_standard assets/characters/animation/male at master · Unity-Technologies/Standard-assets-characters. GitHub. Retrieved January 29, 2023, from https://github.com/Unity-Technologies/Standard-Assets-Characters/tree/master/Assets/_Standard%20Assets/Characters/Animation/Male
[14]YouTube. (2017, December 28). Funny quirky comedy (free download background music). YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=8MgA3SUwFHc
[15]YouTube. (2019, October 11). Unity layers, layer mask, Bitmask, bitwise operators, Raycasts. YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=uDYE3RFMNzk
[16]YouTube. (2020, April 5). [#03] unity animation rigging - attaching a weapon using two bone ik constraint. YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=fB0P0C_3sPU&list=PLyBYG1JGBcd1E4CigRSDE9YdH8syiDY6-&index=9
[17]YouTube. (2020, July 30). [#11] unity animated character controller with root motion. YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=4y4QXEPnkgY&list=PLyBYG1JGBcd1E4CigRSDE9YdH8syiDY6-&index=11
[18]YouTube. (2020, July 8). [#10] weapon reloading using animation events in Unity. YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=QCi2GOyHIIk&list=PLyBYG1JGBcd1E4CigRSDE9YdH8syiDY6-&index=10
[19]YouTube. (2020, June 19). [#09] unity animation rigging: Weapon recoil patterns using overide transform constraint. YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=JD7iDleSzDA&list=PLyBYG1JGBcd1E4CigRSDE9YdH8syiDY6-&index=9
[20]YouTube. (2020, May 17). [#05] shooting a weapon using projectile raycasts (with effects). YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=onpteKMsE84&list=PLyBYG1JGBcd1E4CigRSDE9YdH8syiDY6-&index=5
[21]YouTube. (2020, May 2). [#04] unity animation rigging - aiming a weapon using multi aim constraint. YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=ajmp3J7N3Ow&list=PLyBYG1JGBcd1E4CigRSDE9YdH8syiDY6-&index=4

[22]YouTube. (2021, August 11). Awesome third person shooter controller! (Unity Tutorial). YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=FbM4CkqtOuA&t=1443s
[23]YouTube. (2022, February 14). Local multiplayer and split-screen - new input and Cinemachine. YouTube. Retrieved January 29, 2023, from https://www.youtube.com/watch?v=l9HrraxtdGY&t=418s